



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 08

NOMBRE COMPLETO: Sanchez Villalpando Johan

N° de Cuenta: 422028657

GRUPO DE LABORATORIO: 04

GRUPO DE TEORÍA: 02

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 12/04/25

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para el primer ejercicio se debe agregar un spotlight (que no sea luz de color blanco ni azul) que parta del cofre de su coche y al abrir y cerrar el cofre ilumine en esa dirección. En este caso me guié con la modificación que se le suele hacer a los autos para tener un cofre iluminado por dentro, aunque también modificando el signo de la rotación se puede invertir para que sea una luz hacia afuera.

Para ello primero se crea el spotlight en este caso de color verde.

La posición y dirección se asignará más adelante en base al capó.

```
unsigned int spotLightCount = 0;
//luz fija color verde para cofre
spotLights[0] = SpotLight(0.0f, 1.0f, 0.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    0.0f, 0.0f, 0.0f, //dir
    0.0f, 0.0f, 0.8f,
    50.0); //ángulo
spotLightCount++;
```

Posteriormente en la parte donde va el capó se crea un vector posi que tomará la posición del capó, esta parte de el pivote del capó, por lo que para tener la luz un poco más en medio del capó se usó un posi.y para bajar la luz masomenos a la mitad del capó.

La parte importante consiste en normalizar el vector de la posición junto con la rotación aplicada con las teclas, en el vector direcapo, en este caso se toma en -y para que la luz apunte hacia adentro del capo, ya que si apunta hacia afuera la luz debe rebotar en algo para poder verse bien cuando se mueve.

```
//capo
model = modelaux;
//articulación para el capo
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 3.4f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(-1.0f, 0.0f, 0.0f));
glm::vec3 posi = glm::vec3(model[3]); // posición del objeto
posi.y -= 0.5f;
glm::vec3 direcapo = glm::normalize(glm::vec3(model * glm::vec4(0.0f, 0.0f, -1.0f, 0.0f))); // dirección del capó
```

Se hace uso de setflash para posicionar la luz y que apunte en el ángulo cambiante.

```
spotLights[0].SetFlash(posi, direcapo);
```

Finamente podemos ver como al rotar el capó, la luz también se mueve, adicionalmente al mover el auto, también la luz se mueve pegada al capó.



El siguiente ejercicio consiste en añadir luz de tipo spotlight para el coche de tal forma que al avanzar se encienda una luz y al retroceder se encienda otra.

Para ello se usarán 3 arreglos de spotlights, donde en el primer arreglo tenemos la luz hacia adelante y el cofre (ya que el cofre debe estar siempre encendido según instrucciones), el siguiente arreglo con la luz hacia atrás y el cofre y el tercer arreglo únicamente el cofre.

Para esto primero se definió una variable nueva en las teclas, la cual sirve como bandera para saber si avanza, retrocede o no se está moviendo.

```
avanzando = 0.0f;

//bandera de avance del auto
GLfloat getavanzando() { return avanzando; }
```

La variable avanzando se inicializa en 0.

```
if (key == GLFW_KEY_K)
{
    //Enciende luz azul
    theWindow -> avanzando = 1.0f;
    theWindow->articulacion4 += 0.5;
}
if (key == GLFW_KEY_L)
{
    //Enciende luz roja
    theWindow->avanzando = -1.0f;
    theWindow->articulacion4 -= 0.5;
}
```

Si el auto avanza(k) se pone en 1, si retrocede se pone en -1.

Para lograr el estado donde está quieto se hace uso de GLFW_RELEASE

```
//al soltar k/j deja de avanzar
if (key == GLFW_KEY_K && action == GLFW_RELEASE)
{
    theWindow->avanzando = 0.0f;
}
if (key == GLFW_KEY_L && action == GLFW_RELEASE)
{
    theWindow->avanzando = 0.0f;
}
```

Al detectar que se suelta tanto K, como L, la variable avanzando se coloca en 0, para dar a entender que no hay movimiento ni adelante ni atrás.

Ahora en el código, se crean los 3 arreglos, en este caso yo prefiero llevar un orden de contador por arreglo, sin embargo como los primeros 2 son del mismo tamaño se puede reutilizar un arreglo.

```
//spotlights-----
//cofre y avanza
unsigned int spotLightCount = 0;
//luz fija color verde para cofre
spotLights[0] = SpotLight(0.0f, 1.0f, 0.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    0.0f, 0.0f, 0.0f, //dir
    0.0f, 0.0f, 0.8f,
    50.0); //ángulo
spotLightCount++;
//luz fija color azul para avanzar
spotLights[1] = SpotLight(0.0f, 0.0f, 1.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    -1.0f, 0.0f, 0.0f, //dir
    0.9f, 0.0f, 0.0f,
    30.0f); //ángulo
spotLightCount++;
//cofre y retrocede-----
unsigned int spotLightCount2 = 0;
spotLights2[0] = SpotLight(0.0f, 1.0f, 0.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    0.0f, 0.0f, 0.0f, //dir
    0.0f, 0.0f, 0.8f,
    50.0); //ángulo
spotLightCount2++;
//luz fija color rojo para retroceder
spotLights2[1] = SpotLight(1.0f, 0.0f, 0.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    1.0f, 0.0f, 0.0f, //dir
    0.8f, 0.0f, 0.0f,
    30.0f); //ángulo
spotLightCount2++;
//solo cofre.
unsigned int spotLightCount3 = 0;
spotLights3[0] = SpotLight(0.0f, 1.0f, 0.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    0.0f, 0.0f, 0.0f, //dir
    0.0f, 0.0f, 0.8f,
    50.0); //ángulo
spotLightCount3++;
```

Finalmente se utilizan condicionales para los 3 casos, donde además se agrega ahora el posicionamiento de la luz del cofre y el movimiento de las luces azul y roja con respecto al auto.

```
//inicia control de arreglos.  
//avanza  
if (mainWindow.getavanzando() == 1.0f) {  
    shaderList[0].SetSpotLights(spotLights, spotLightCount);  
    spotLights[0].SetFlash(posi, direcapo);  
    spotLights[1].SetPos(glm::vec3(-6.0f - mainWindow.getarticulacion4(), 0.0f, 0.0f)); //luz azul avanza  
}  
//retrocede  
else if (mainWindow.getavanzando() == -1.0f) {  
    shaderList[0].SetSpotLights(spotLights2, spotLightCount2);  
    spotLights2[0].SetFlash(posi, direcapo);  
    spotLights2[1].SetPos(glm::vec3(6.5f - mainWindow.getarticulacion4(), 0.0f, 0.0f)); //luz roja retrocede  
}  
//quieto  
else {  
    shaderList[0].SetSpotLights(spotLights3, spotLightCount3);  
    spotLights3[0].SetFlash(posi, direcapo);  
}
```

Tenemos el resultado:



Para el último ejercicio se crea otra luz de tipo puntual ligada a otro modelo distinto al de la lampara, ambos se deben poder controlar.

Para esto primero se definen las teclas, las cuales son recicladas de prácticas anteriores en mi caso.

```
if (key == GLFW_KEY_Z)
{
    theWindow->articulacion5 = 0.0;
}
if (key == GLFW_KEY_X)
{
    theWindow->articulacion5 = 1.0;
}
if (key == GLFW_KEY_C)
{
    theWindow->articulacion6 = 0.0f;
}
if (key == GLFW_KEY_V)
{
    theWindow->articulacion6 = 1.0f;
}
```

articulacion5 apaga con Z y enciende con X, articulacion6 apaga con c y enciende con v.

Para hacerlo simple utilicé solamente valores de 0/1, aunque podría hacerlo booleano.

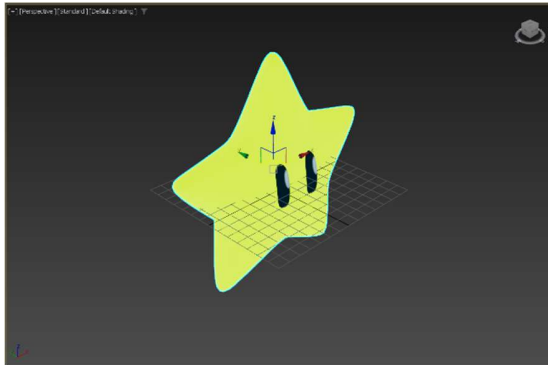
Se crean los 3 arreglos, el primero con lampara encendida únicamente, el segundo con la estrella(mi modelo) encendida únicamente y el tercero con ambos encendidos, igual que en la spot se puede optimizar reutilizando contadores si se requiere.

```
//pointlights-----
//Lampara encendida
unsigned int pointLightCount = 0;
//Declaración de primer luz puntual
pointLights[0] = PointLight(1.0f, 1.0f, 1.0f, //color blanco
    0.7f, 0.3f, //radiación, saturación.
    5.0f, 1.25f, -2.5f, //posición
    0.3f, 0.2f, 0.1f); //valores de la ecuación de segundo grado
pointLightCount++;

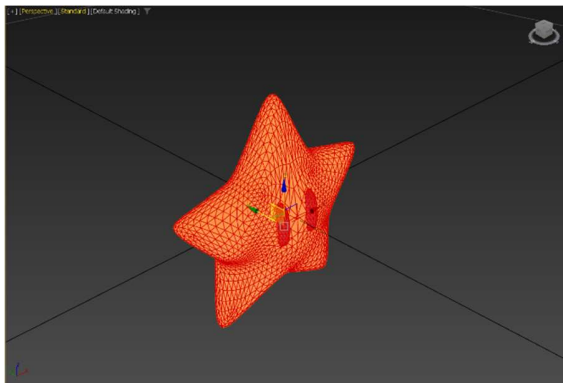
//estrella encendida segundo arreglo
unsigned int pointLightCount2 = 0;
pointLights2[0] = PointLight(1.0f, 1.0f, 0.0f, //color amarillo
    0.7f, 0.5f, //radiación, saturación.
    5.0f, 5.0f, 2.5f, //posición
    0.3f, 0.2f, 0.2f); //valores de la ecuación de segundo grado
pointLightCount2++;

//ambas encendidas tercer arreglo
unsigned int pointLightCount3 = 0;
//Declaración de primer luz puntual
pointLights3[0] = PointLight(1.0f, 1.0f, 1.0f, //color blanco
    0.7f, 0.3f, //radiación, saturación.
    5.0f, 1.25f, -2.5f, //posición
    0.3f, 0.2f, 0.1f); //valores de la ecuación de segundo grado
pointLightCount3++;
//estrella encendida
pointLights3[1] = PointLight(1.0f, 1.0f, 0.0f, //color amarillo
    0.7f, 0.5f, //radiación, saturación.
    5.0f, 5.0f, 2.5f, //posición
    0.3f, 0.2f, 0.2f); //valores de la ecuación de segundo grado
pointLightCount3++;
```

Para el modelo elegí una estrella de Mario Bros.



Se toma el modelo y se trianguliza, además de que se reduce el tamaño para hacerlo adecuado, el pivote ya estaba ajustado.



Se añade al programa.

```
//lampara
Model lampara;
//estrella
Model estrellita;

//estrella
estrellita = Model(); //Creamos modelo
estrellita.LoadModel("Models/estrellita.obj");
```

En el bucle se crean las condicionales.

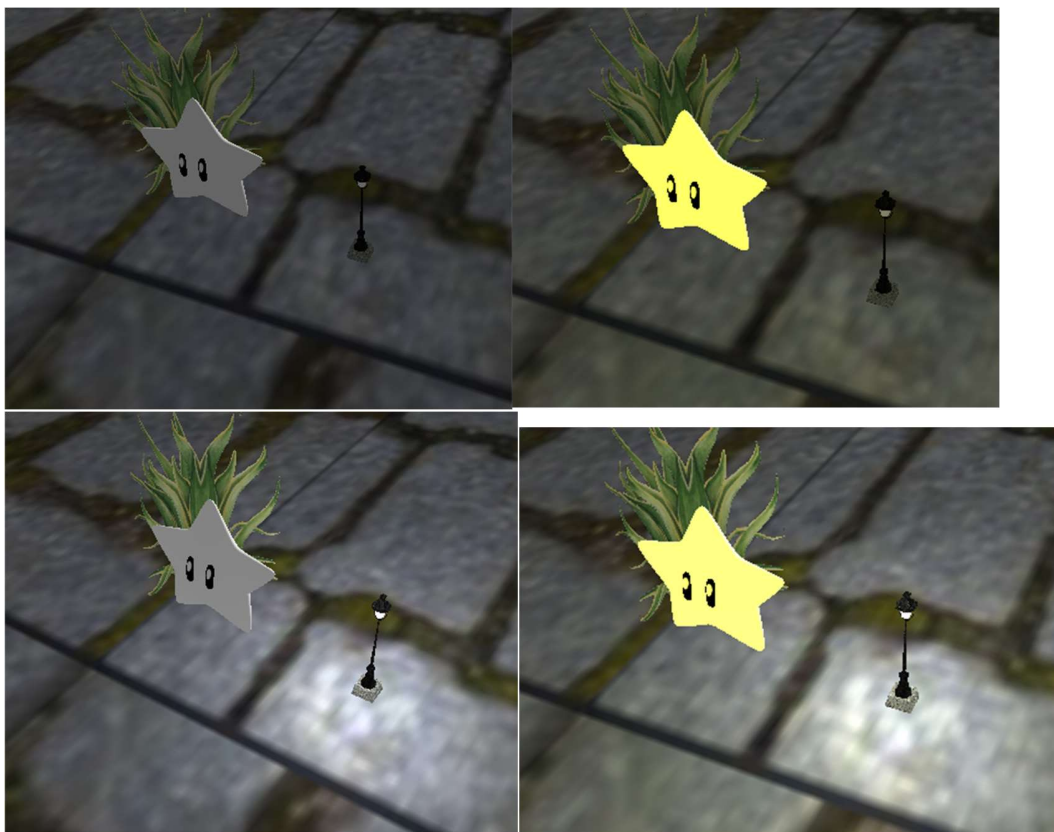
```

//inicia control de arreglos luces puntuales.
//lampara
if (mainWindow.getarticulacion5() == 1.0f && mainWindow.getarticulacion6() == 0.0f) {
    shaderList[0].SetPointLights(pointLights, pointLightCount);
}
//estrella
else if (mainWindow.getarticulacion5() == 0.0f && mainWindow.getarticulacion6() == 1.0f) {
    shaderList[0].SetPointLights(pointLights2, pointLightCount2);
}
//lampara y estrella
else if (mainWindow.getarticulacion5() == 1.0f && mainWindow.getarticulacion6() == 1.0f) {
    shaderList[0].SetPointLights(pointLights3, pointLightCount3);
}
//ninguna
else {
    //un pointligh vacío
    shaderList[0].SetPointLights(pointLights, 0);
}

```

Para el caso de ambas apagadas, se usa un arreglo de luces de tamaño 0 para no dibujar ninguna luz.

Finalmente se tienen ambas luces.



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Para esta práctica no hubo mucho problema más que el planteamiento de cómo debe ir la luz del capó ya que no entendía si era hacia arriba o hacia abajo, así que me base en modificaciones reales que se le suelen hacer a los autos.

Para las luces no hubo problema en los arreglos, ya que la bandera de avanzando fue muy útil y simplificó todo ahorrándome usar condicionales más complejas.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Esta práctica fue interesante ya que empezamos a darle más uso a las luces junto con las teclas y junto con el movimiento de posición y de ángulo, heredando aspectos de los modelos, considero que fue divertida y me da ideas de lo que se puede hacer en el proyecto final, considerando el límite de luces con el que trabajamos pienso que se tiene que planear muy bien cómo se usarán estas luces.

La explicación fue buena y entendí cómo se debía realizar la práctica, la idea de normalizar fue un poco confusa pero finalmente quedó.

Bibliografía en formato APA

- Marc Mons. (15 Sep 2024). *Calle de la vieja farola modelo 3d*. Turbosquid.com. Recuperado el 01 de abril de 2025, de <https://www.turbosquid.com/es/3d-models/3d-old-lamp-street-model-2281618#>
- *What does it mean to normalize a value?* (s/f). Stack Overflow. Recuperado el 11 de abril de 2025, de <https://stackoverflow.com/questions/11644356/what-does-it-mean-to-normalize-a-value>
- Sketchfab. (10 Oct 2022). super mario star - Download Free 3D model by gaelinix.

Recuperado el 11 de abril de 2025 de <https://sketchfab.com/3d-models/super-mario-star-cd988ff787734dec955cd76e8acdf327>